



Chat hooks

Reuse headless request lifecycles and chat-interface behavior.

Problem and boundary

Public contract

Working integration

@epam/ai-dial-chat-hooks



When this library is useful

Reuse headless request lifecycles and chat-interface behavior.

01

Build a custom chat

Reuse stream state, scrolling and attachment behavior.

02

Keep rendering separate

Connect returned data and callbacks to your own UI.

03

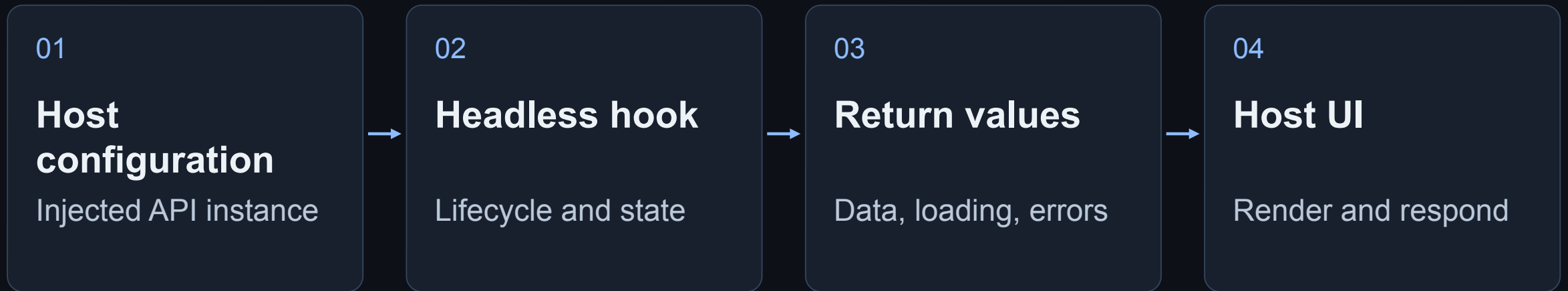
Limit imports

Use the dependency-focused public subpaths.



Follow the ownership boundary

Inputs and effects stay explicit.



Library: Reusable hook state, data mapping and permitted thin operation wrappers.



The public contract to start with

useShareLink · selected surface; full API index is in the bundle.

Input or operation	Contract
useViewportWidth()	number; browser resize lifecycle
useShareLink(api, id, origin)	data, isLoading, error, setAccess
Injected API	Pick<ShareApi, createShareLink>
Subpath entries	Focused public import surfaces



A minimal workspace integration

TypeScript / React example · imports and host setup abridged.

```
function ViewportDemo() {  
  const width = useViewportWidth();  
  return <p>Demo viewport: {width}px</p>;  
}  
// Import from /viewport-layout.  
// The hook removes its resize listener on cleanup.
```

Full example: [examples/chat-hooks.tsx](#) · uses [@epam/ai-dial-chat-hooks](#)



Wire a realistic host workflow

Keep chat hooks focused on its public responsibility.

01

Keep the client stable

Inject a configured ShareApi operation into useShareLink.

02

Render the lifecycle

Read data, isLoading and error from the hook result.

03

Change link access

Call setAccess to request a link for the new access level.



A request lifecycle with an injected client

The caller configures auth and transport.

```
const { data, isLoading, error, setAccess } =  
  useShareLink(  
    api,  
    'demo-item',  
    'https://example.com',  
  );  
// Render data.url, pending state or the error.
```

Focused source-backed example · see [examples/](#) and [content/api-index.md](#).



Available customization

01

Small layout entry

`/viewport-layout`
includes
`useViewportWidth` with
listener cleanup.

02

Request lifecycle

`useShareLink` manages
link loading, access
changes and stale
response guards.

03

Broad feature entries

Conversation, files,
catalog, OAuth and
MCP have explicit
subpaths.



Find the integration and its checks

Existing tests are evidence of intended behavior; suite results are not claimed.

Evidence	Where to look
Real consumer	<code>toolset-editor/src/components/AuthSection/AuthSection.tsx</code>
Test evidence	selects a file and resets the input so the same file can be selected again
Test evidence	refreshes once when the active conversation is missing without looping on item changes



Constraints that affect integration

01

Stable dependencies

Recreating API instances can retrigger effects.

02

Browser assumptions

Some hooks use browser globals; pass origin explicitly for share examples.

03

Boundary audit needed

Some current helpers contain more DIAL-specific knowledge than the narrow rule.